

Quantum Hybrid Research & Optimization Lab

This document provides a detailed, implementation-focused description of the Quantum Hybrid Research & Optimization Lab. The Lab is a Motia-orchestrated platform for designing, executing, and analyzing hybrid quantum-classical algorithms across optimization, simulation, and machine learning use cases, with disciplined comparison to classical baselines.

1. Problem & Vision

Organizations exploring quantum computing face a recurring set of issues:

- Experiments scattered across notebooks, scripts, and vendor consoles.
- Hard-to-reproduce results tied to specific hardware snapshots and environments.
- Limited comparability between quantum approaches and strong classical baselines.
- No cohesive interface to plug in real optimization or simulation problems from production systems.

The Quantum Hybrid Research & Optimization Lab addresses this by:

- Treating quantum workloads as governed, first-class workflows in a shared platform.
- Providing clear mappings from domain problems to quantum-friendly formulations.
- Unifying simulators and QPUs behind a consistent execution and logging layer.
- Enabling rigorous, side-by-side evaluation so quantum claims can be validated quantitatively.

2. Core Outcomes

The Lab is designed to achieve:

1) Structured Quantum Experimentation

- Every run defined as code + config + environment, not as an untracked notebook cell.

2) Domain-Ready Problem Templates

- Reusable encodings for portfolio optimization, routing, scheduling, max-cut, and small chemistry models.

3) Honest Benchmarking

- Built-in classical baselines and error mitigation so results are credible.

4) Automated Exploration

- Optuna/OpenEvolve-driven search spaces over ansatz depth, mixers, optimizers, and learning rates.

5) Knowledge Retention

- Central registry of experiments so teams can query "what worked where and why" instead of restarting from scratch.

3. Architecture Overview

Motia orchestrates all major flows: problem registration, circuit generation, execution, mitigation, baseline comparison, and reporting.

Key components:

- Problem Definition Layer:

- SDK + API for defining optimization/simulation tasks with metadata.
- Mappers that translate these into QUBO/Ising or Hamiltonian operators.

- Quantum Backend Layer:

- Qiskit, PennyLane, Cirq, and QuTiP for expressing and simulating circuits.

- Configurable routing to select between local simulators and external QPUs.
- Error Mitigation Layer:
 - Mitiq plugged into the execution path (ZNE, PEC, CDR) when enabled.
- Classical Baseline Layer:
 - PyTorch/JAX plus OR-tools/MILP/heuristics for strong baselines.
- Storage & Registry:
 - PostgreSQL for metadata and access control.
 - Object storage for artifacts (circuit dumps, logs, plots, pickled models).
 - MLflow for experiment, metric, and artifact tracking.
- Retrieval & Assistant:
 - OpenSearch + Qdrant index experiment configs, logs, and commentary.
 - Optional assistant to help design new runs based on previous outcomes.

4. Key Workflows

4.1 Define & Register Problem

- Researcher or engineer submits a problem spec (e.g., N-asset portfolio with constraints).
- The system validates the input and encodes it into canonical forms.
- Problem definitions are versioned and tagged (domain, size, constraints).

4.2 Quantum Experiment Execution

- User selects problem + algorithm template (QAOA, VQE, QSVM, etc.).
- Circuit Generator builds the circuit family for given hyperparameters.
- Execution Orchestrator dispatches runs to selected backends, capturing noise models when available.
- Results are stored with full lineage: backend, parameters, seeds, calibration context.

4.3 Error Mitigation & Robustness

- Configurable toggle attaches Mitiq to runs for mitigation.
- Outputs include side-by-side mitigated vs raw results, enabling robustness analysis.

4.4 Classical Baseline & Comparative Evaluation

- Classical solvers run on the same problem to produce reference solutions.
- Comparison reports quantify optimality gaps, runtime ratios, scaling behavior, and sensitivity.

4.5 HPO & Evolutionary Search

- Optuna/OpenEvolve operate as outer loops around quantum experiment workflows.
- Objective functions can include: best-found cost, stability across seeds, depth/shot budgets.
- Search history and Pareto fronts are logged to MLflow.

4.6 Experiment Registry & RAG

- All experiments, configs, logs, and notes are indexed.
- Users can ask: "Show QAOA performance for routing problems of size 20-40" or "Which ansatz families failed to beat classical baselines?" and retrieve structured answers.

5. Security, Governance & Integration

- Role-based access control to projects and experiments.
- Tenant separation for different business units or external collaborators.
- Optional integration with the Secure AI & Data Governance Control Plane so that:
 - Data sources used in quantum workloads are policy-checked.
 - Logs and decisions feed enterprise SIEM and compliance views.
- Designed so sensitive proprietary datasets remain governed while being used for quantum research.

6. Implementation & Extensibility

Phase 1: Core experiment registry, problem definition layer, and local simulator support.

Phase 2: Integration with Qiskit/PennyLane/Cirq backends and MLflow tracking.

Phase 3: Add Mitiq, HPO/evolution loops, and automated comparative evaluation.

Phase 4: Onboard external QPU providers, domain-specific templates, and tighter integration with other platforms (financial lab, logistics, RWA analytics).

The Lab is intentionally opinionated about reproducibility, while remaining flexible on backends and algorithms. It becomes the internal "OS" for quantum experiments that can be demonstrated to stakeholders with hard metrics, not hype.